웹서비스설계 실습 02

검색·정렬·페이지네이션 게시판 개선하기

01 실행 환경 및 범위

학번: 202413123 **이름:** 조이진 **제출일:** 2026년 11월 5일

GitHub 저장소 URL: https://github.com/zohijin-k/notice_board

본 실습은 Node/Express 서버를 실행하고 `http://localhost:3000`에서 동작을 확인하는 방식으로 진행합니다. 설치와 실행 명령은 `npm install`, `npm start`이며, `npm start`는 `node server.js`와 동일합니다.

서버는 실행할 때마다 1,000건의 인메모리 데이터를 생성하고 조회수는 무작위로 부여합니다. 따라서 조회수와 응답 시간은 실행마다 달라질 수 있습니다. 응답 시간은 인메모리 배열 처리의 관찰값이므로, 실제 DB 인덱스의 시간복잡도나 성능을 나타내지 않습니다.

02 기존 코드와 직접 수정한 부분

수정은 서버의 검색·정렬·입력 검증, 화면의 페이지네이션·비동기 처리, 실행 및 제출 안내의 세 영역에서 이루어졌습니다.

[2-1] 검색·정렬 (`server.js`)

- **커서 검색:** 기존 커서 API는 `search`를 처리하지 않았습니다. 제목과 본문 검색 조건을 추가하고, 커서 조건과 함께 적용한 뒤 `id DESC` 정렬과 `limit + 1` 확인을 수행하도록 변경했습니다.
- **정렬 화이트리스트:** 기존 화이트리스트는 일반 객체였습니다. null-prototype 객체와 `Object.hasOwn`으로 허용 키만 통과시켜, `__proto__`, `constructor`, `toString`과 임의 값은 `latest`로 처리되도록 했습니다.
- **입력 검증:** 기존 `parseInt`는 `12abc` 같은 값을 부분적으로 숫자로 인정할 수 있었습니다. 양의 정수 문자열을 엄격히 검사하고, 잘못된 입력은 기본값으로 처리하도록 했습니다.

[2-2] 페이지네이션 및 비동기 화면 (`index.html`)

- **커서 모드:** 정렬을 비활성화하고 ID 내림차순 고정 안내를 표시했습니다. 검색어를 커서 API URL에 전달하고, 마지막 페이지에서는 더 보기 버튼을 비활성화합니다.
- **검색 입력:** 입력 시 현재 요청을 즉시 취소하고 페이지, 커서, 누적 목록을 초기화한 뒤 300ms 후 새 검색을 요청합니다.
- **응답 경합 방지:** 기존 AbortController 취소에 요청 순번 확인을 더해, 완료 직전의 오래된 응답이 최신 화면을 덮지 못하게 했습니다. `AbortError`는 정상 취소로 처리합니다.
- **SQL 영역 안내:** 우측 SQL 영역이 실제 DB/SQL 실행 결과가 아니라 인메모리 동작 설명용 미리보기임을 명시했습니다.

[2-3] 실행 및 제출 안내 (`package.json`, `README.md`)

- `npm start` 스크립트를 추가했습니다.
- 설치, 실행, 개념 설명, 변경 전후, AI 활용 내역을 README에 정리했습니다.

03 구현 검증

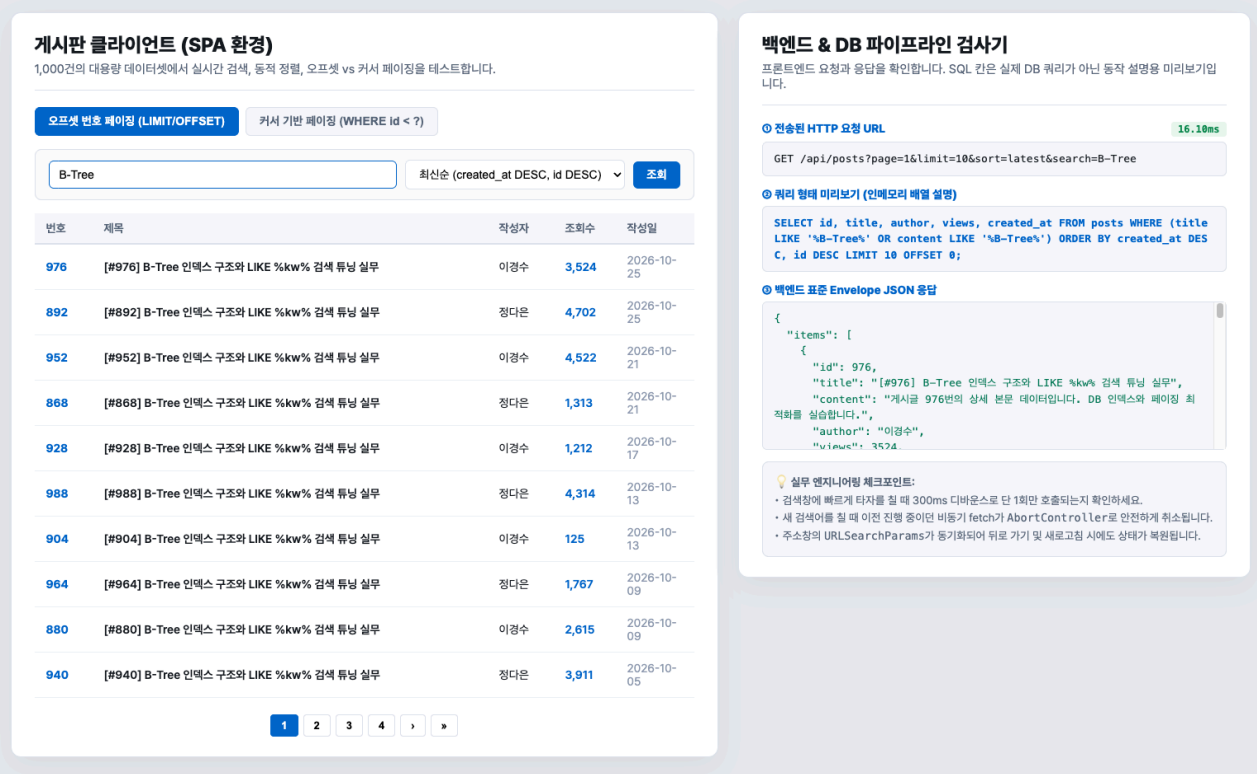
[3-1] 검색·정렬

구현 설명

- matchesSearch()가 검색어를 소문자로 정규화한 뒤 게시글의 title 또는 content에 포함되는지 검사합니다.
- 검색 결과가 없으면 items: [], totalCount: 0을 응답하고 프론트엔드가 검색 결과 없음 안내를 표시합니다.
- 정렬 키는 latest, oldest, views만 허용합니다. SORT_WHITELIST는 null prototype 객체이며 Object.hasOwn()으로 키를 검사해 __proto__, constructor 등 상속 속성을 허용 키로 취급하지 않습니다.
- 허용 목록에 없는 정렬 키는 latest 비교 함수를 사용합니다.
- 최신순은 작성일 내림차순, 오래된순은 작성일 오름차순입니다. 조회수순은 조회수 내림차순이며 동점 게시글은 ID 내림차순으로 정렬합니다.

항목별 테스트 및 캡처

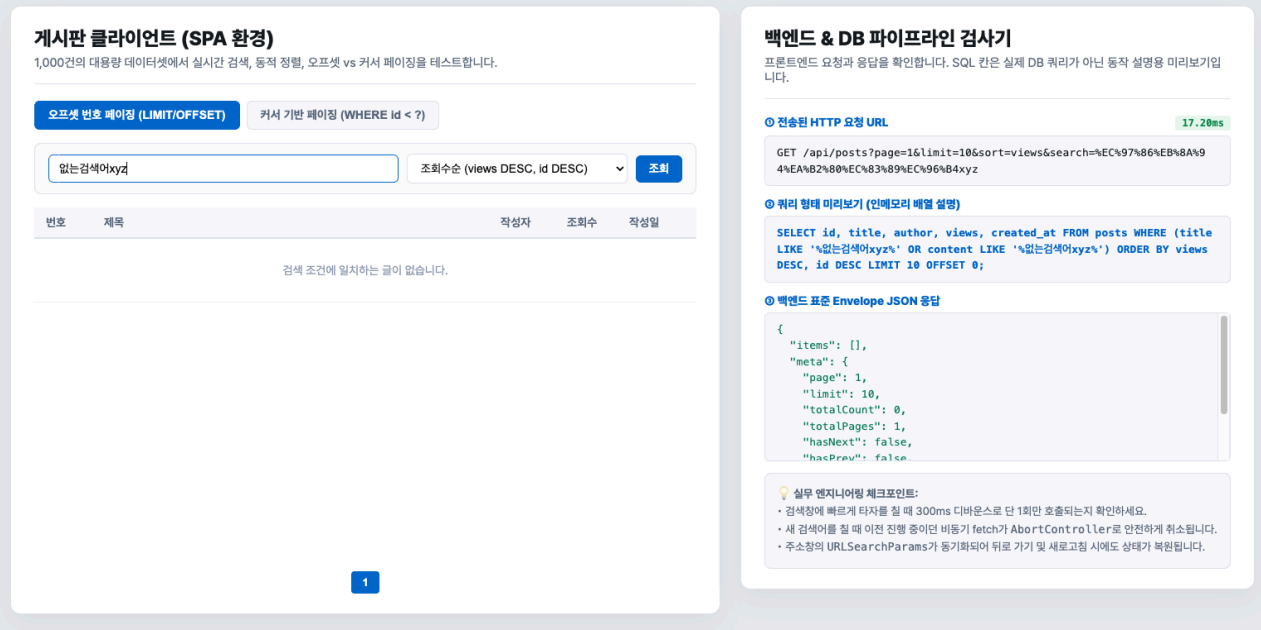
검증 조건	입력·방법	test 결과
제목 검색	검색창에 B-Tree 입력	제목에 B-Tree가 있는 여러 게시글이 반환되고 요청 URL에 검색어가 포함됩니다.



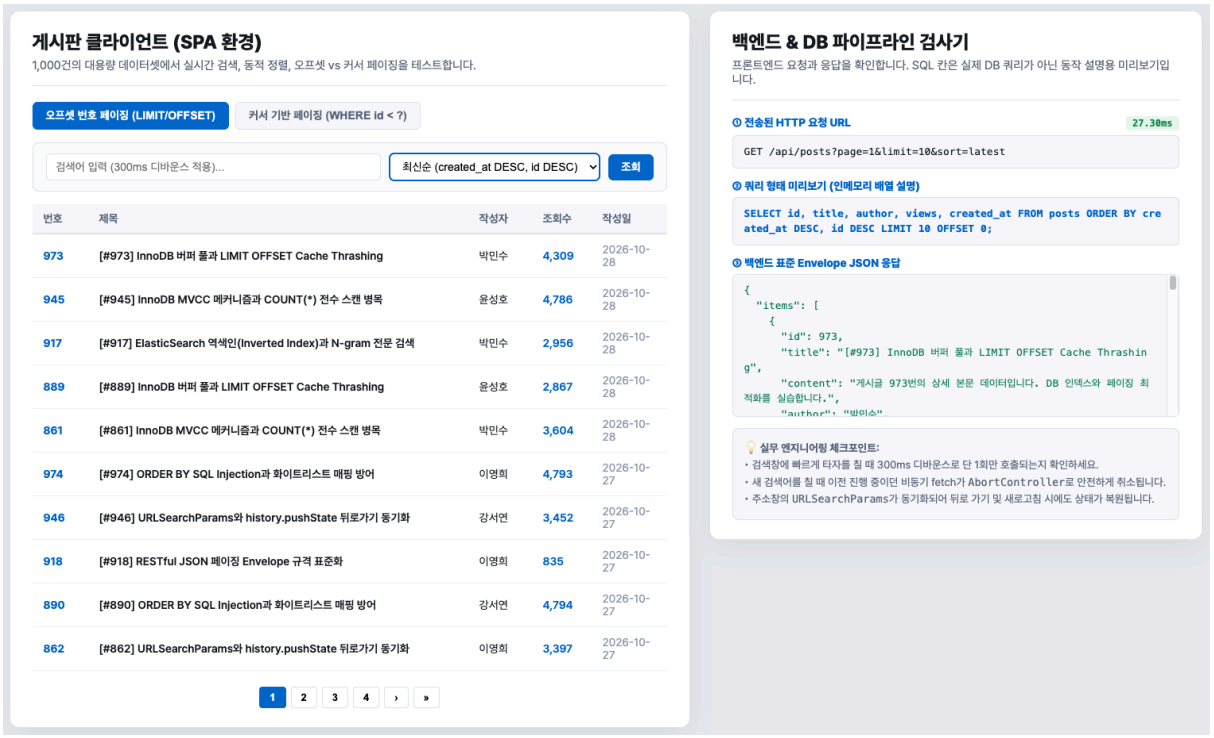
본문 검색	검색창에 게시글 1000번 입력	제목에 검색어가 없어도 본문에 해당 문자열이 있는 게시글 1건이 반환됩니다. 응답 JSON의 content에서 검색어를 확인할 수 있습니다.



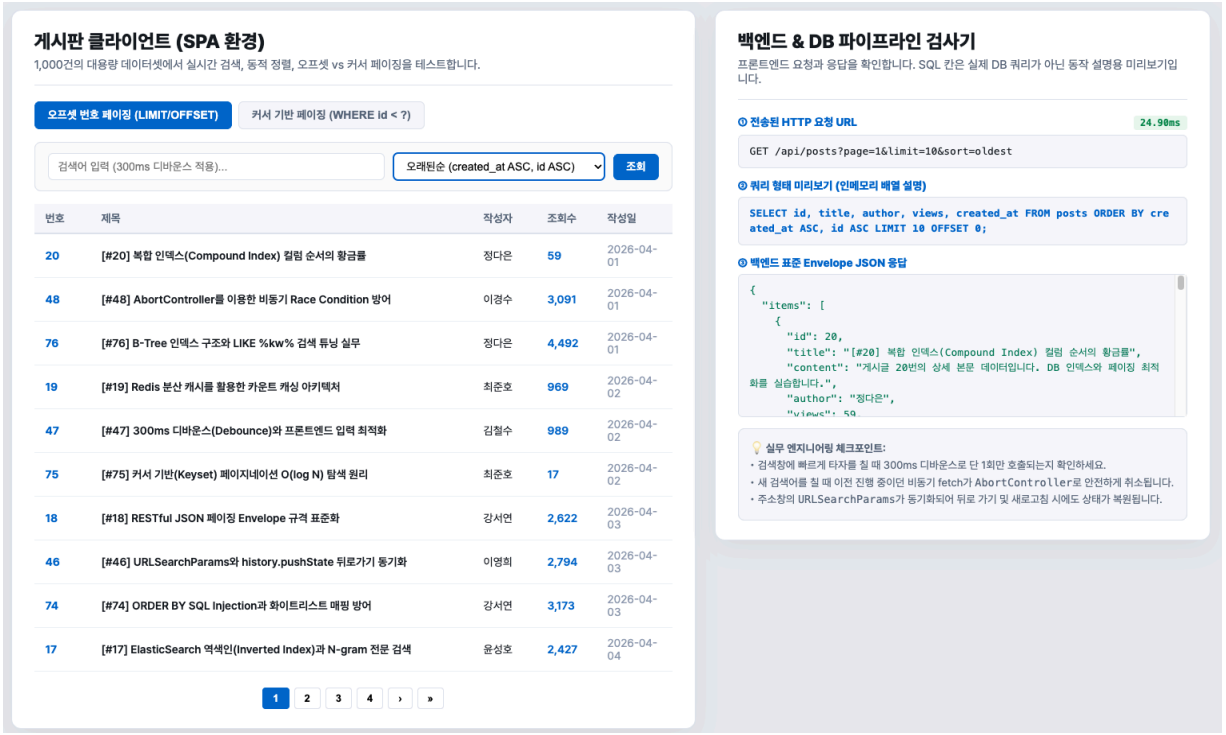
검색 결과 없음	없는검색어xyz 입력	게시글이 없고 items: [], totalCount: 0, 검색 결과 없음 안내가 표시됩니다.



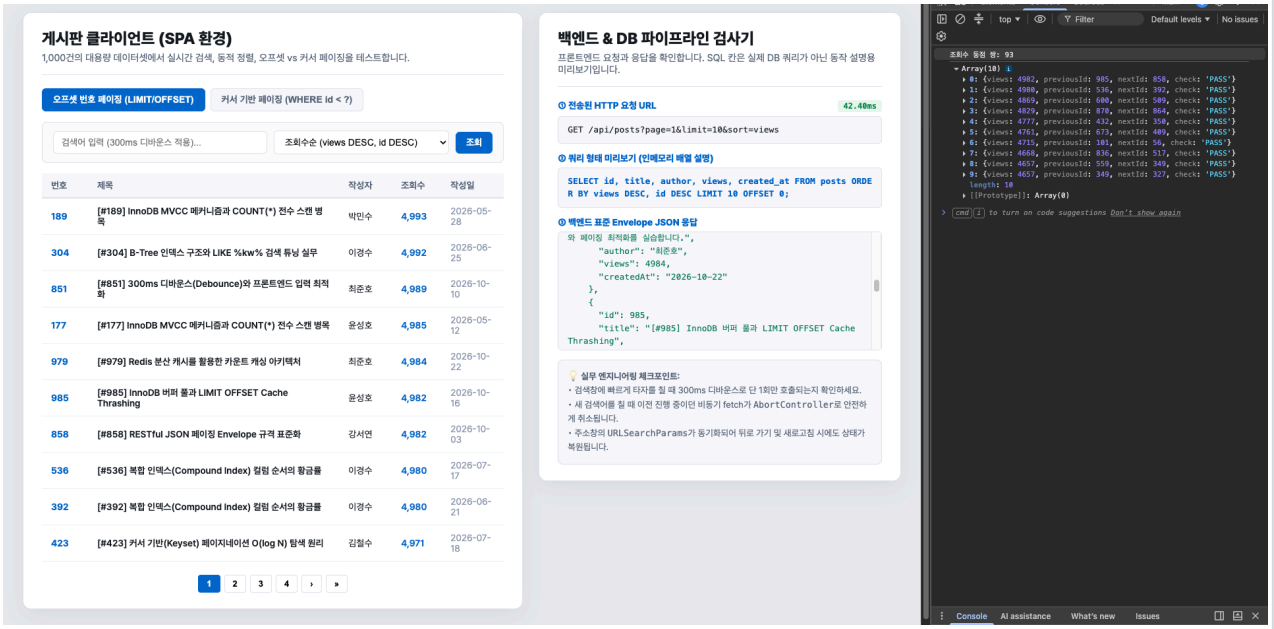
최신순	latest 선택	작성일이 최신 날짜부터 표시됩니다.



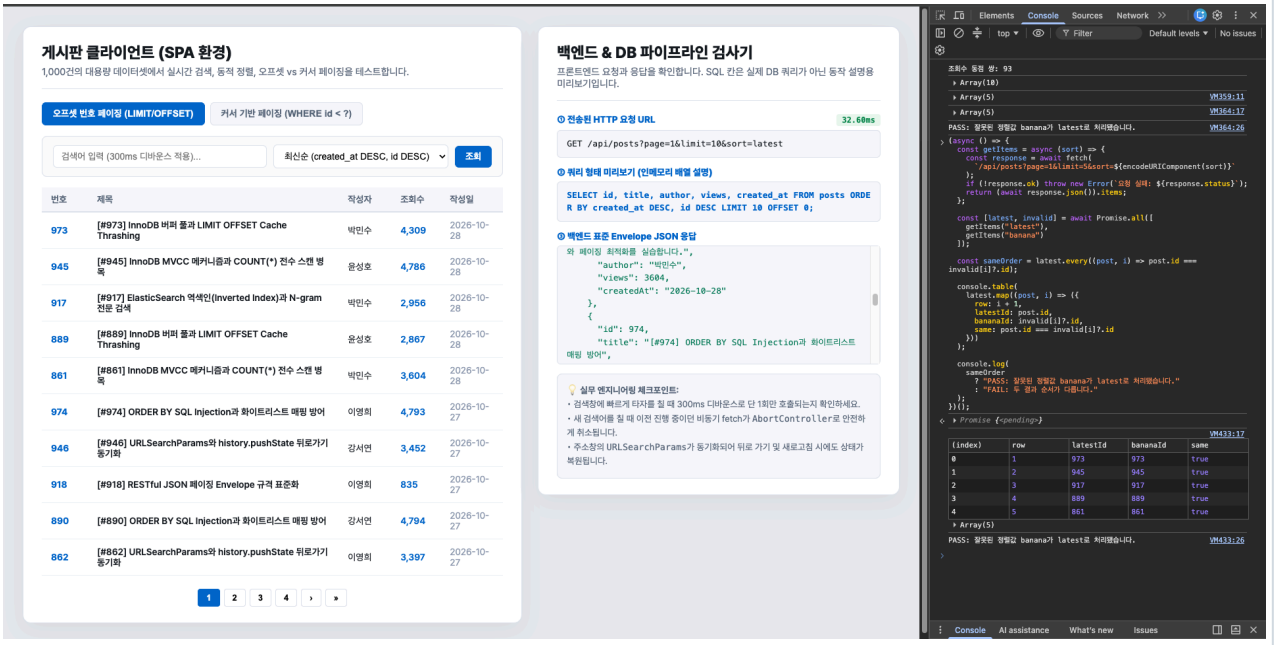
오래된순	oldest 선택	작성일이 오래된 날짜부터 표시됩니다. 첫 날짜는 2026-04-01입니다.



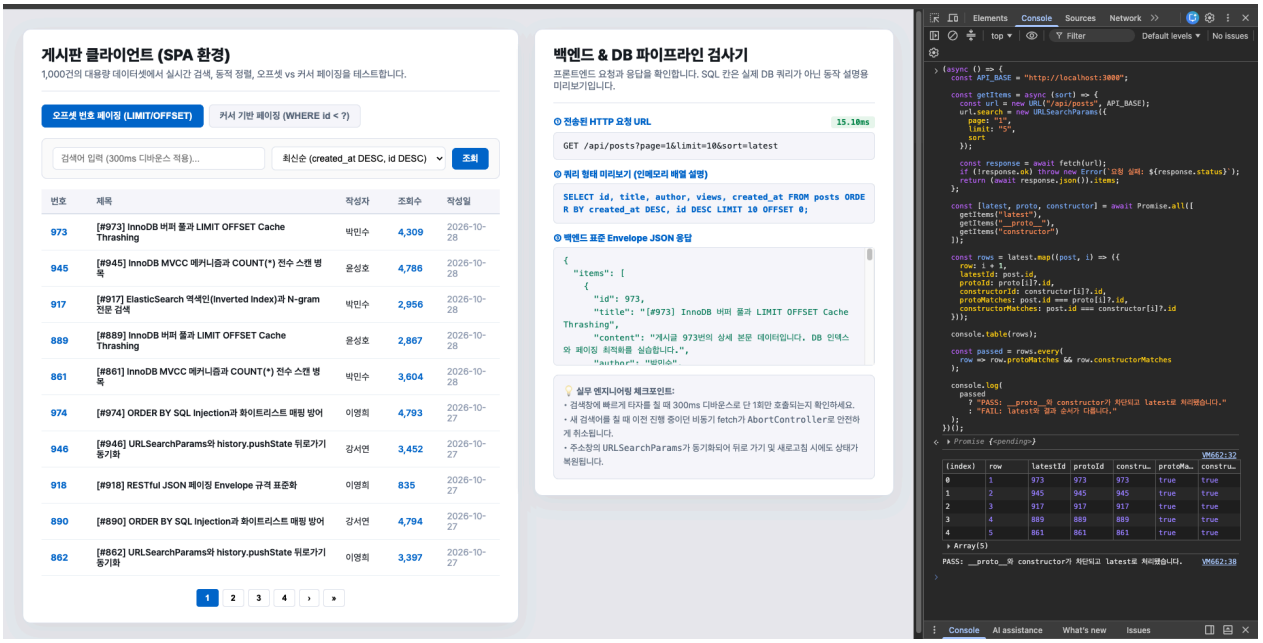
조회순 및 동점 ID	views 선택 후 Console에서 동점 ID 비교	조회수가 내림차순이며, 동점인 93쌍 모두 ID 내림차순 검사 결과 PASS입니다.



허용되지 않은 값의 기본 처리	sort=banana와 latest 결과 ID 비교	두 요청의 ID 순서가 같고 Console에 잘못된 정렬 키가 latest로 처리됐다는 PASS가 출력됩니다.



상속 속성 차단	sort=__proto__, sort=constructor 비교	두 값 모두 허용된 정렬 키로 동작하지 않으며 최신순 결과와 ID가 같습니다. Console에 PASS가 표시됩니다.



[3-2] 페이지네이션

구현 설명

- 오프셋 API GET /api/posts는 검색과 정렬을 먼저 적용한 다음 현재 페이지 구간을 반환합니다.
- 커서 API GET /api/posts/cursor도 제목·본문 검색을 지원합니다. 검색 조건과 id < cursor를 적용한 뒤 ID 내림차순으로 정렬하고, 그 결과에서 limit + 1개를 확인합니다.
- 커서 모드는 ID 내림차순으로 고정됩니다. UI에서 정렬 선택 메뉴를 비활성화하고 고정 기준을 안내합니다.
- 검색어가 바뀌면 프론트엔드는 기존 cursor와 누적 목록을 비우고 새 검색의 첫 요청부터 시작합니다.
- hasNext가 false면 더 보기 버튼을 비활성화하고 완료 문구를 표시합니다.

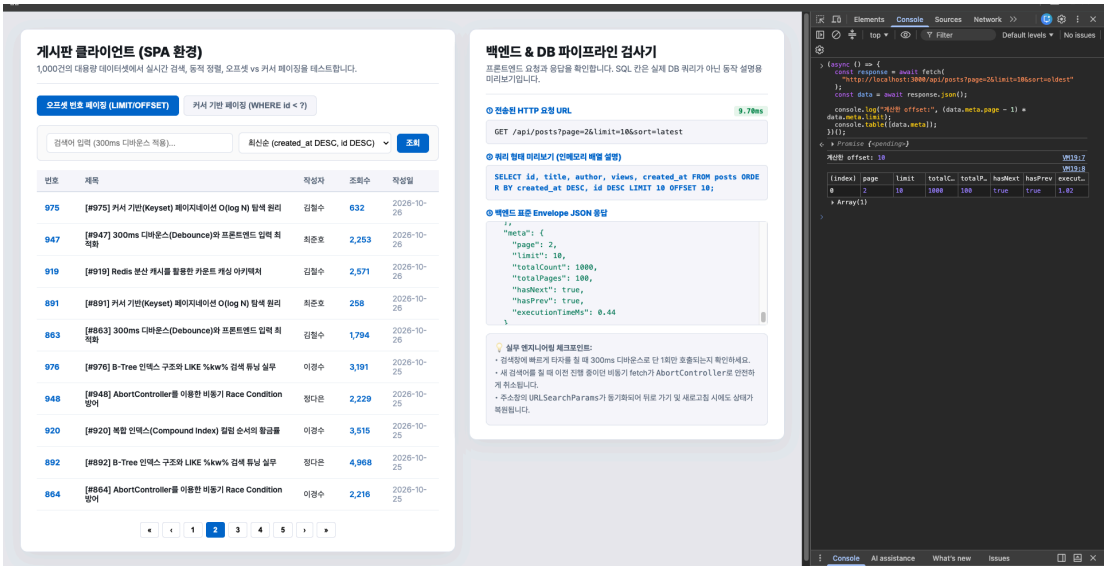
오프셋 계산과 응답 meta

offset = (page - 1) × limit
totalPages = max(1, ceil(totalCount ÷ limit))

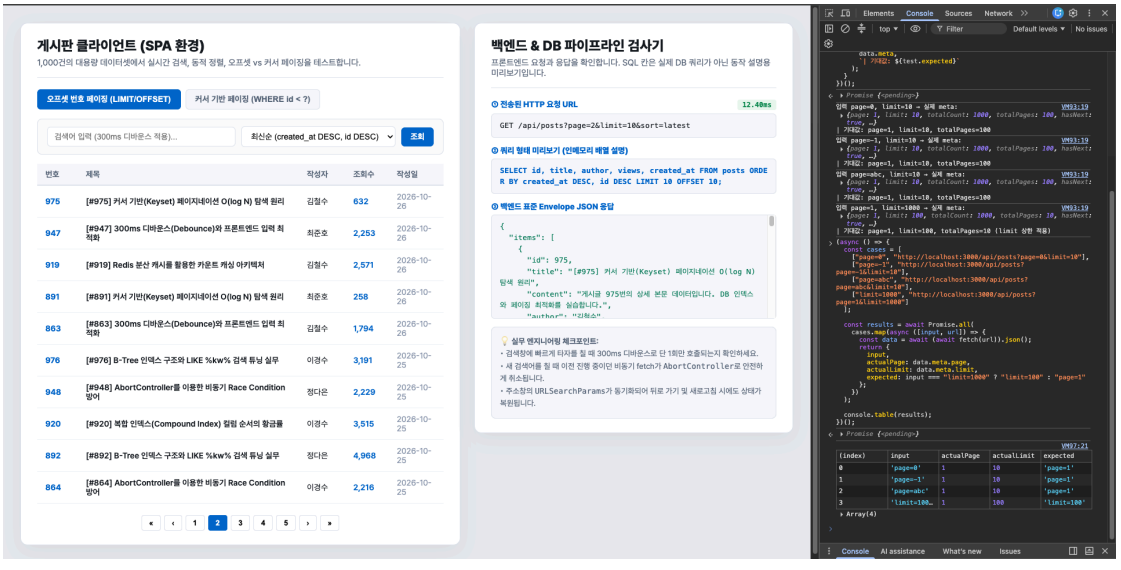
page=2, limit=10이면 offset=10으로, 정렬된 검색 결과의 11번째부터 20번째까지 가져옵니다. totalPages는 전체 검색 결과 수를 페이지 크기로 나눈 값을 올림한 것입니다. 검색 결과가 0건일 때도 페이지 수가 최소 1로 유지됩니다.

항목별 테스트 및 캡처

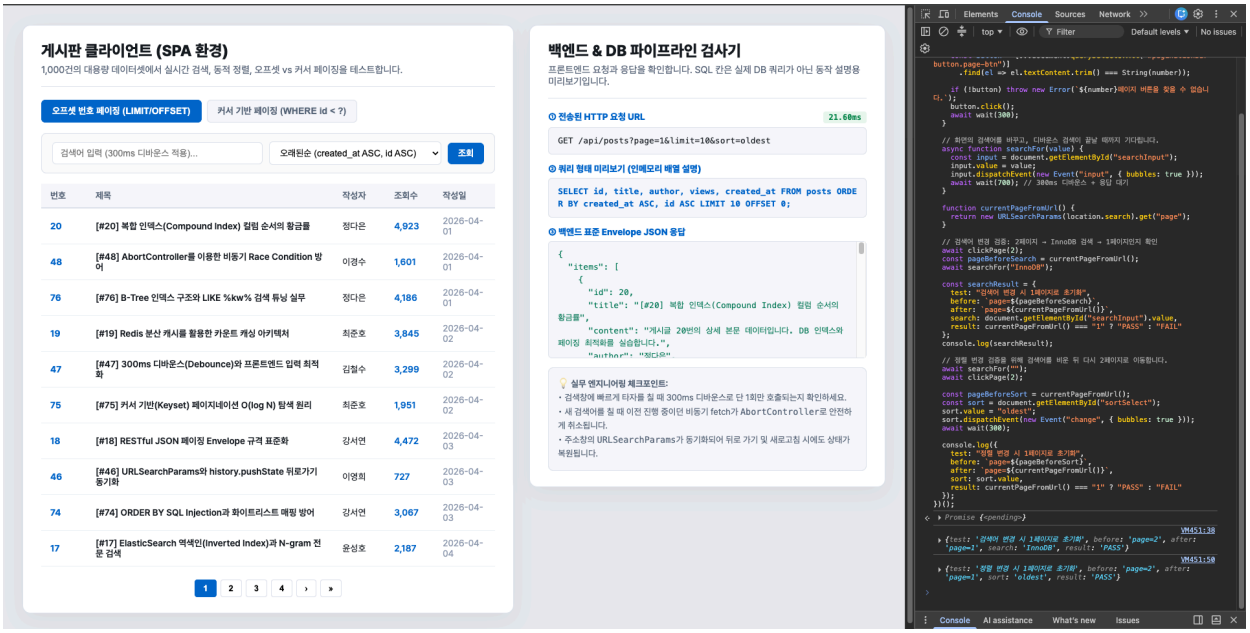
검증 조건	입력·방법	test 결과
오프셋 페이지 이동·meta	page=2&limit=10&sort=oldest 요청	응답 meta에서 page=2, limit=10, totalCount=1000, totalPages=100, hasPrev=true, hasNext=true를 확인했습니다. offset=10은 SQL 형태 미리보기에서 확인했습니다.



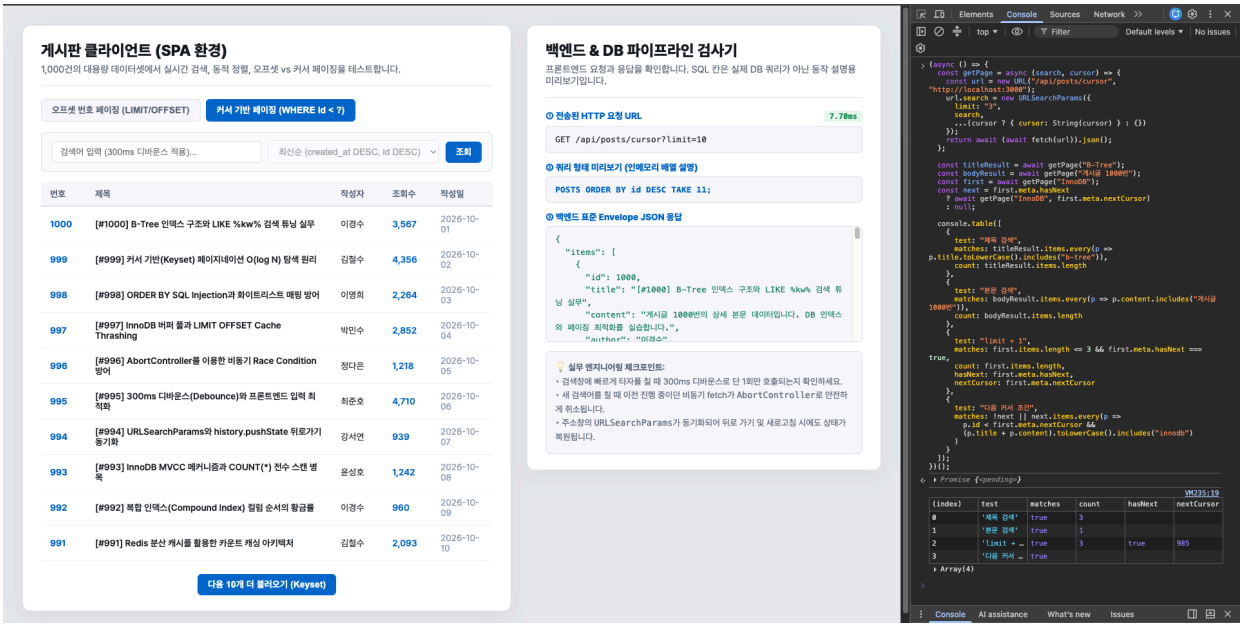
잘못된 page·limit	page=0, -1, abc, limit=1000 요청	세 잘못된 page 값은 page=1로, limit=1000은 상한 100으로 보정됩니다. 데이터 1,000건 기준 totalPages=10입니다.



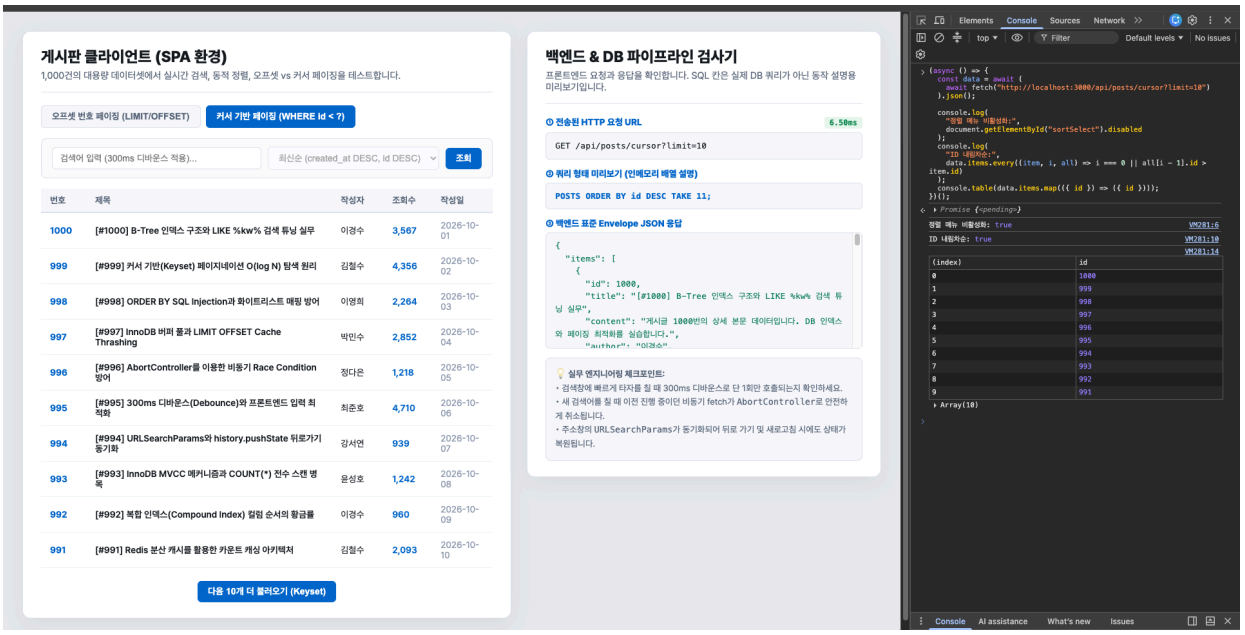
검색·정렬 변경 후 초기화	2페이지에서 검색어를 변경하고, 다시 2페이지에서 정렬을 변경	두 경우 모두 URL의 page가 1로 바뀌고 Console 검사가 각각 PASS입니다.



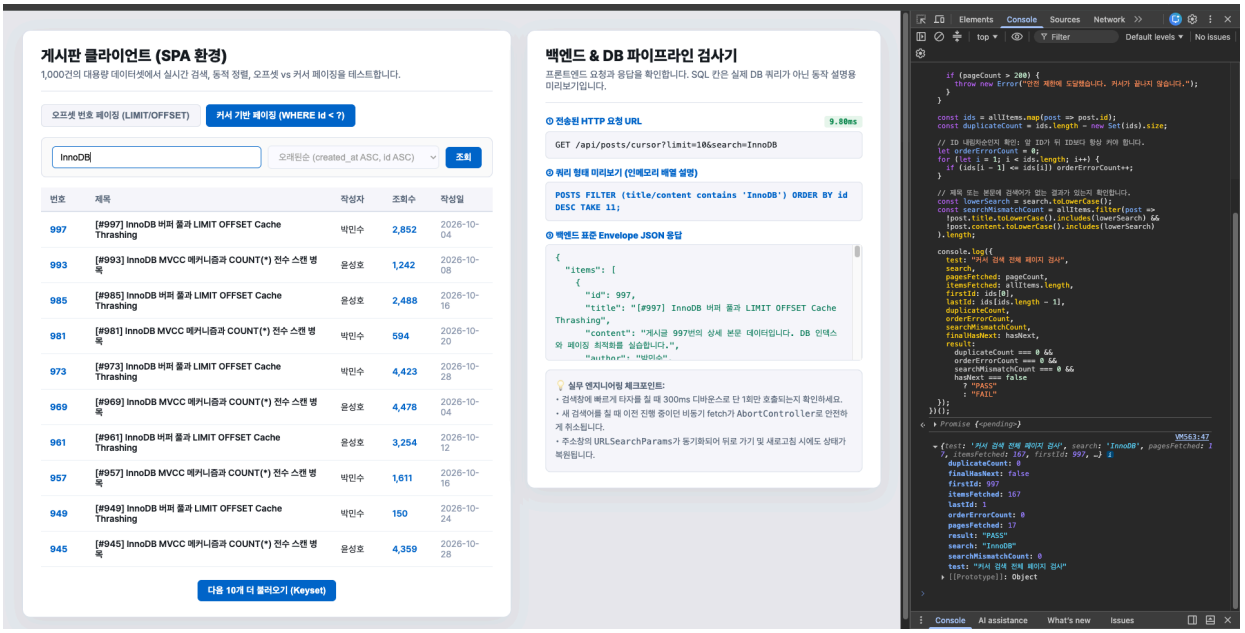
커서 제목·본문 검색 및 검색 이후 limit + 1	제목 B-Tree, 본문 게시글 1000번 검색 후 다음 커서 요청	두 검색 모두 일치 결과를 반환하고 다음 커서 요청에도 검색어 조건이 이어집니다. limit 3 검사에서 다음 데이터가 있어 hasNext=true와 nextCursor를 확인했습니다. 화면 미리보기의 기본 limit 10은 TAKE 11입니다.



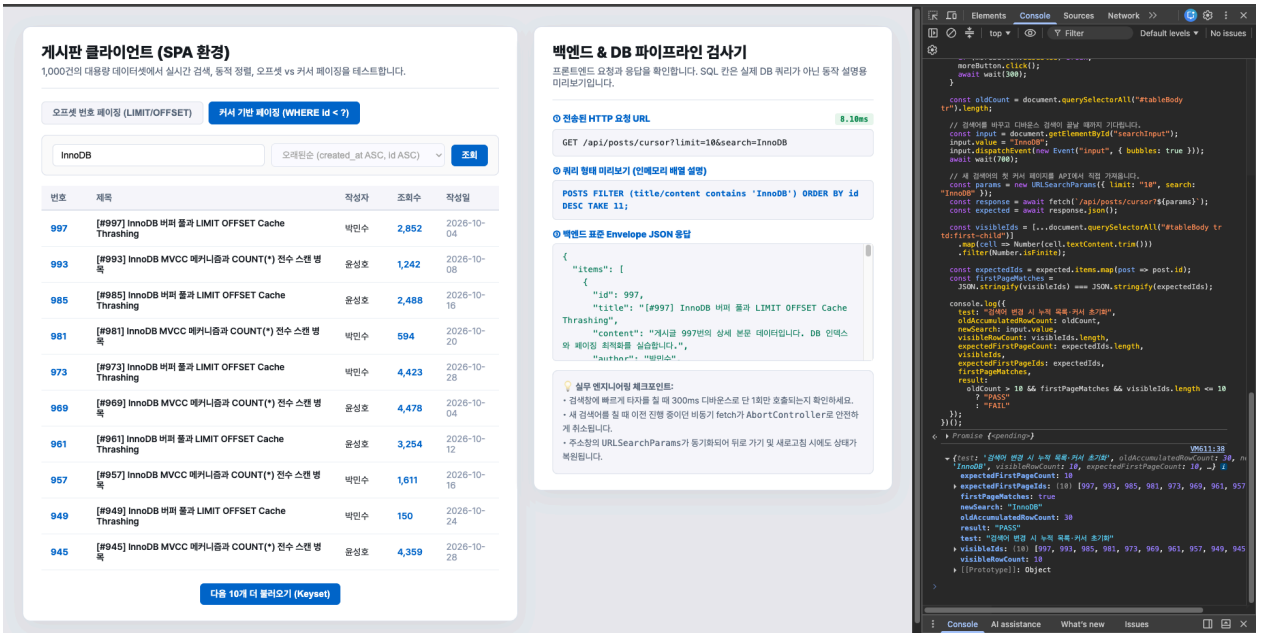
ID 내림차순·정렬 메뉴 비활성화	커서 탭 선택 후 Console에서 ID와 disabled 확인	ID 1000부터 991까지 감소하고 정렬 메뉴 disabled 검사가 true입니다.



끝까지 조회·연속성 검사	InnoDB 검색을 cursor로 마지막까지 반복 요청	17회 요청에서 167건을 수집했습니다. 중복 0, 순서 오류 0, 검색 불일치 0, 마지막 hasNext=false, Console PASS입니다.



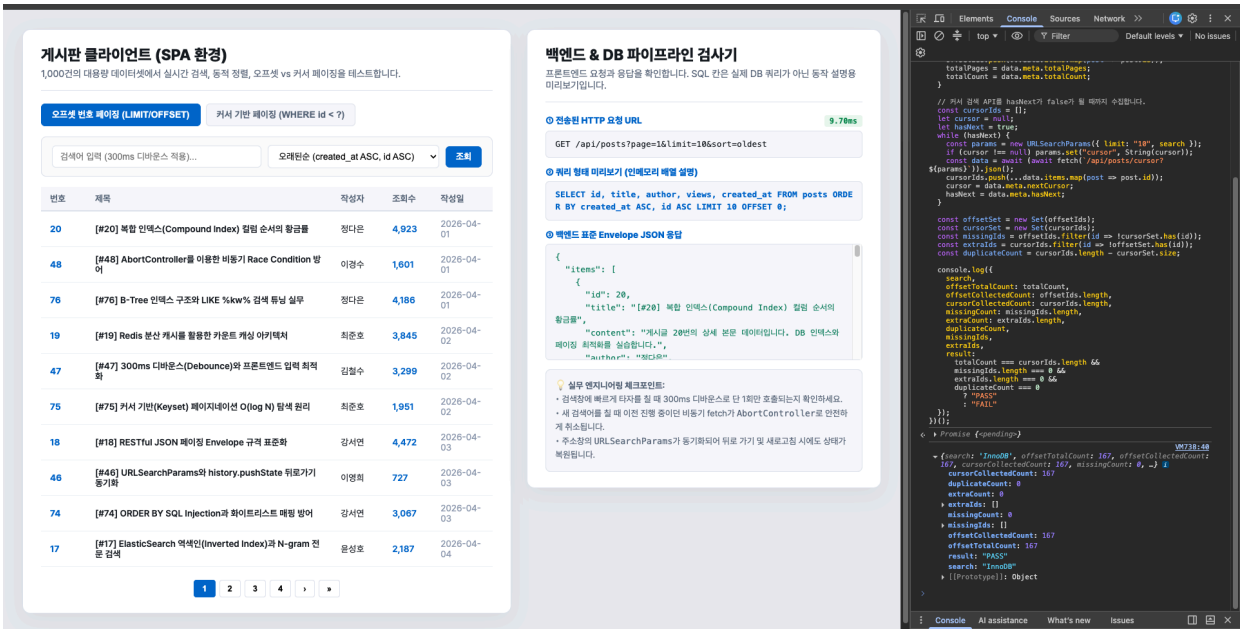
검색어 변경 시 목록·커서 초기화	기존 결과를 30건 누적한 뒤 InnoDB 검색	새 결과 10건이 새 검색의 첫 API 페이지 ID와 일치하고 PASS입니다. 이전 검색의 누적 목록은 이어지지 않습니다.



마지막 페이지 버튼 상태	게시글 1000번 검색	결과 1건, hasNext=false, 더 보기 버튼 disabled, Console PASS입니다.



누락·추가·중복 대조	오프셋 검색 결과와 전체 커서 결과의 ID 집합 비교	양쪽 모두 167건이며 누락 0, 추가 0, 중복 0, Console PASS입니다. 표시 정렬 순서가 다를 수 있으므로 ID 집합으로 비교했습니다.



[3-3] 비동기 처리

구현 설명

- 검색 입력 이벤트마다 기존 debounce 타이머를 지우고 300ms 타이머를 새로 설정합니다. 입력이 이어지는 동안에는 요청을 보내지 않고, 입력이 멈춘 뒤 마지막 검색어로 요청합니다.
- 요청마다 AbortController를 사용합니다. 다음 입력이나 요청이 발생하면 앞선 요청을 abort합니다.
- 요청 sequence 번호를 함께 검사해 오래된 요청의 응답이 늦게 도착하더라도 최신 결과 화면을 덮어쓰지 않게 합니다.
- AbortError는 일반 오류와 구분해 안전하게 처리합니다. 그 외 오류만 실제 fetch 오류로 기록합니다.
- 응답 시간은 브라우저에서 관찰된 값이며, 캡처마다 달라질 수 있습니다.

항목별 테스트 및 캡처

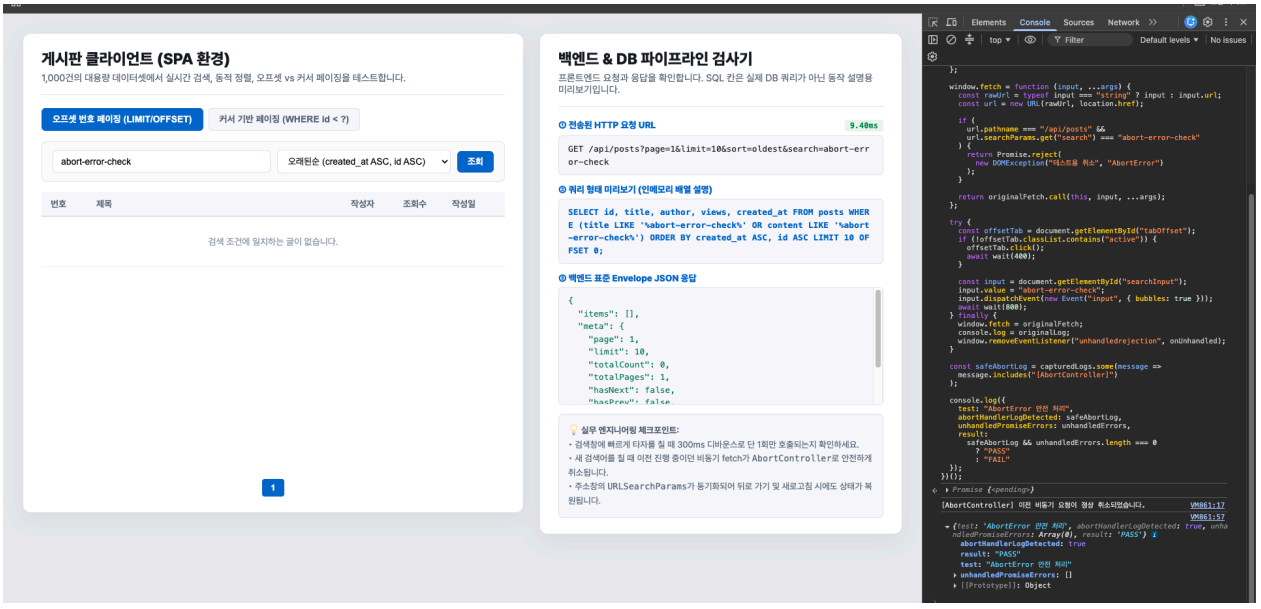
검증 조건	입력·방법	test 결과
300ms 디바운스	입력 이벤트 6회를 100ms 간격으로 발생시킴	300ms보다 짧은 간격으로 입력했을 때 마지막 InnoDB 요청 1회만 기록되고 PASS입니다.



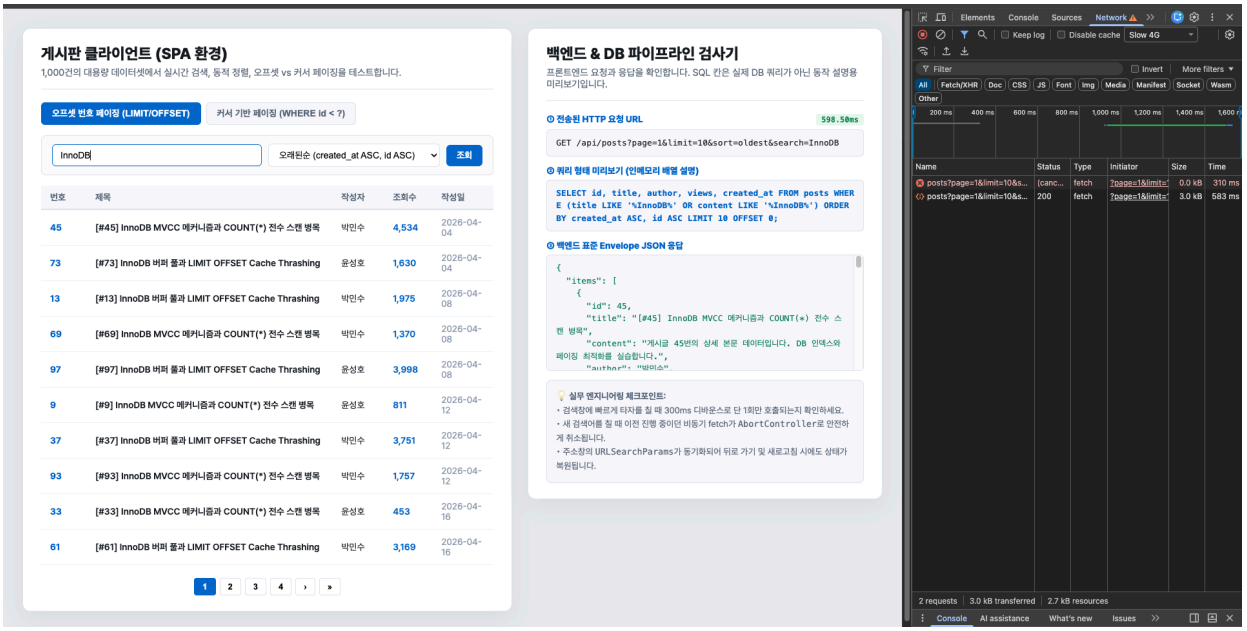
AbortController 신호·늦은 응답 방지	B-Tree 응답을 지연시키고 그 사이 InnoDB 검색	이전 요청의 signal.aborted가 true이고, 지연된 이전 응답이 도착해도 화면은 InnoDB 결과를 유지합니다. 지연 응답 경합을 재현하는 시험에서는 첫 fetch의 signal 전달을 차단해 오래된 응답 방지 동작을 별도로 확인했습니다. 실제 네트워크 요청 취소는 다음 행의 Slow 4G 시험으로 확인했습니다.



AbortError 안전 처리	특정 요청에서 테스트용 AbortError를 발생시킴	취소 로그가 감지되고 처리되지 않은 Promise 오류가 없어 PASS입니다.



느린 네트워크 검색 변경·요청 취소	Network 탭을 Slow 4G로 설정하고, 검색 요청이 진행되는 동안 검색어를 변경	Network 목록에 취소된 이전 fetch와 완료된 최종 200 요청이 표시됩니다. 검색창과 결과는 최종 검색어 InnoDB에 일치합니다. 응답 시간은 화면 관찰값이며 고정 성능 지표가 아닙니다.

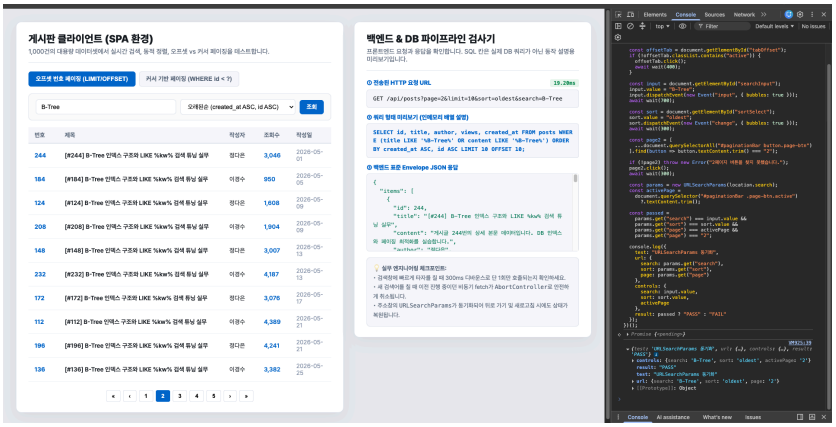


[3-4] URL 상태·복원

구현 설명

- 오프셋 요청 상태는 URLSearchParams로 page, limit, sort, search를 직렬화해 URL에 반영합니다.
- 검색·정렬·페이지 이동은 history.pushState()로 히스토리 항목을 만듭니다.
- 새로고침 때 URL의 파라미터를 읽어 검색창과 정렬 메뉴를 채우고 해당 페이지 결과를 요청합니다.
- popstate 이벤트에서도 URL을 다시 읽고 UI 입력값과 목록을 복원합니다.

항목별 테스트 및 캡처

검증 조건	입력·방법	test 결과
URLSearchParams 동기화	B-Tree 검색, oldest 선택, 2페이지 이동	URL의 search=B-Tree, sort=oldest, page=2가 입력창·정렬 선택·활성 페이지와 일치하고 PASS입니다.
		
새로고침·뒤로 가기 복원	저장된 2페이지 상태에서 새로고침 후 뒤로 가기	새로고침 후 2페이지 상태, 뒤로 가기 후 1페이지 상태로 복원됩니다. 두 상태의 URL·검색창·정렬·응답 결과가 일치하고 전체 PASS입니다.
		

04 페이지네이션 개념 정리

① 오프셋과 커서는 어떤 화면에 적합한가

- 오프셋 방식은 페이지 번호를 직접 선택하거나 전체 페이지 수를 보여주는 게시판·관리 화면에 적합합니다. 원하는 페이지로 바로 이동하기 쉽습니다. 대신 뒤쪽 페이지로 갈수록 앞의 많은 행을 건너뛰어야 할 수 있고, 조회 도중 데이터가 추가·삭제되면 페이지 경계에서 항목이 밀리거나 반복될 수 있습니다.
- 커서 방식은 피드나 무한 스크롤처럼 다음 항목을 이어서 보여주는 화면에 적합합니다. 직전 결과의 마지막 ID를 다음 기준으로 사용하므로 큰 페이지 번호를 계산하지 않아도 됩니다. 임의 페이지로 바로 이동하거나 전체 페이지 수를 표시하는 사용 사례에는 덜 적합합니다.
- 이 예제의 커서 정렬 키는 ID 내림차순으로 고정합니다. 다음 요청은 id < nextCursor 조건을 사용해 직전 페이지보다 오래된 ID를 가져옵니다.

② 디바운스와 요청 취소의 역할

- 디바운스는 입력이 잠시 멈출 때까지 기다려 연속 타이핑 중 발생하는 요청 횟수를 줄입니다. 300ms 디바운스는 마지막 입력 후 300ms 동안 추가 입력이 없을 때 검색을 요청합니다.
- 요청 취소는 이미 보낸 요청이 진행 중인 상태에서 검색어가 바뀌었을 때 이전 요청의 작업을 중단하도록 합니다.
- 두 기능은 목적이 다릅니다. 디바운스는 요청을 보내기 전 호출 빈도를 줄이고, AbortController는 이미 진행 중인 요청을 취소합니다. 요청 sequence 검사는 취소가 늦거나 응답이 경합하는 상황에서 오래된 결과가 화면을 덮는 것을 막습니다.

③ limit + 1 원리

- 화면에 limit개만 보여주더라도 서버가 후보 결과에서 한 건을 추가로 확인하면 다음 항목이 있는지 알 수 있습니다. 후보가 limit + 1개 이상이면 hasNext=true로 설정하고, 실제 응답에서는 여분 1개를 제외합니다. 후보가 limit개 이하라면 hasNext=false입니다. 커서 방식은 이 방법으로 다음 페이지 존재 여부를 판단하며 전체 건수 조회를 하지 않습니다.

④ offset과 totalPages 계산식

```
offset = (page - 1) × limit
totalPages = max(1, ceil(totalCount ÷ limit))
```

- 예를 들어 page=2, limit=10이면 offset=10입니다. totalCount=1000, limit=10이면 totalPages=100입니다. totalCount=0일 때는 화면에서 페이지 1을 유지할 수 있도록 totalPages의 최소값을 1로 둡니다.

05 AI 활용

AI를 사용해 서비스 구현을 완료하고, 검증 가이드 및 README 초안 작성을 요청했습니다.